

MCA Semester – IV

Project – Final Report

A study on “NovaBank: A Secure Full-Stack Digital Banking Web Application with AI-Driven Fraud Detection and Financial Health Analytics”

A Project submitted to Jain Online (Deemed-to-be University) in partial fulfillment of the requirements for the award of:

Master of Computer Applications

Name	Chethan BV
USN	241VMTR01537
Elective	Full Stack Development
Date of Submission	08/08/2026

Under the guidance of:

Faculty-JAIN Online
(Faculty-JAIN Online)

Jain Online (Deemed-to-be University)
Bangalore

DECLARATION

I, Chethan BV, hereby declare that the Project Report titled “NovaBank: A Secure Full-Stack Digital Banking Web Application with AI-Driven Fraud Detection and Financial Health Analytics” has been prepared by me under the guidance of Faculty-JAIN Online. I declare that this Project work is towards the partial fulfilment of the University Regulations for the award of the degree of Master of Computer Application by Jain University, Bengaluru. I have undertaken a project for a period of one semester. I further declare that this Project is based on the original study undertaken by me and has not been submitted for the award of any degree/diploma from any other University / Institution.

Place: Bangalore

Date: 08/08/2026

Name of the Student: Chethan BV

USN: 241VMTR01537

ACKNOWLEDGEMENT

I express my deep gratitude to my project guide and the academic team at Jain Online (Deemed-to-be University) for their continuous encouragement, technical guidance, and structured feedback throughout the lifecycle of this Capstone project.

I am also thankful to the open-source community for providing standard, reliable frameworks—specifically the Spring Boot ecosystem, React ecosystem, and MySQL database engine—which made it possible to build this application to industrial standards.

Finally, I extend my appreciation to my peers and family for their support, which kept me motivated to complete this full-stack development journey and successfully deliver the final implementation.

EXECUTIVE SUMMARY

NovaBank is a production-grade, secure, full-stack digital banking web application engineered to address key functional, security, and analytical gaps in mid-scale financial platforms. Developed as a Capstone project in Full Stack Development, this project integrates modern user interface design with a robust transaction processing system, rule-based artificial intelligence for fraud risk assessment, and data-driven financial health insights.

The application architecture follows a layered MVC modular design pattern. The front-end is built using React 18 and Tailwind CSS for single-page responsiveness, utilizing React Router v6 for secure client-side routing, and Axios interceptors for automated token attachment. The backend service layer is developed using Java 17 and Spring Boot 3, employing Spring Security 6 for authentication, stateless JSON Web Token (JWT) verification, and Spring Data JPA with MySQL 8.0 for data persistence. Concurrent transactions are synchronized at the database level using pessimistic write locks to prevent double-spending and ensure ACID transactional integrity.

A key innovation in NovaBank is the inclusion of intelligent services: a real-time Fraud Detection Engine that analyzes transaction parameters (e.g. limit violations, rapid successive payments, and newly added beneficiary accounts) to assign risk scores, and a Financial Health Service that calculates savings metrics and delivers personalized advisory tips to customers. This document presents the complete system architecture, database model, security mechanisms, user interface designs, and evaluation results, verifying that all capstone requirements have been fully implemented.

TABLE OF CONTENTS

Title	Page Nos.
Executive Summary	1
1. Introduction	1
2. System Architecture	4
3. Technologies Used	7
4. Front-end Development	9
5. Back-end Development	11
6. Database Design	14
7. Authentication and Security	16
8. Project Management	18
9. Results and Evaluation	20
10. Conclusion	22
Annexure A. Key Implemented REST Endpoints	23
Annexure B. Entity-to-Module Mappings	24
Annexure C. System Design diagrams	25

1. Introduction

The digital banking domain has transitioned from physical branches to digital-first customer service models, where users demand instant, secure, and intuitive web access. While large financial institutions utilize proprietary systems, mid-scale platforms often struggle with fragmented experiences, inadequate security, lack of transaction consistency safeguards, and an absence of proactive fraud monitoring.

NovaBank is designed to address these limitations. The project implements a robust digital banking portal supporting two primary roles: Customer and Administrator. Customers can perform account management, profile updates, beneficiary management, and fund transfers. Administrators have access to centralized metrics, transaction history, customer profiles (excluding credentials), and real-time fraud logs.

The significance of the project lies in its practical application of advanced full-stack techniques, including DTO-based API design, stateless JWT validation, atomic double-entry bookkeeping ledger operations, pessimistic database locking, and automated fraud scoring logic. This project serves as an integrated demonstration of software engineering principles in web development.

2. System Architecture

NovaBank utilizes a client-server MVC architecture. The React frontend handles user interactions and local state management, while the Spring Boot backend acts as a stateless REST API, exposing endpoints for business operations. Data storage is isolated in a normalized MySQL relational database.

The backend is structured into distinct, decoupled packages: controllers handle HTTP requests and map DTO payloads; services contain transaction-safe business logic; repositories manage database transactions via Spring Data JPA; entities define the relational database mappings; config packages manage application parameters; security packages implement filter chains and JWT tokens; and exception packages handle errors centrally.

Communication is secured via HTTP over TLS. When a customer logs in, a JWT is issued by the backend. The React app stores this token and includes it in the Authorization header of subsequent requests. An Axios request interceptor handles token insertion, while route guards (ProtectedRoutes) prevent unauthenticated users from accessing protected views.

Figure 1. Context Diagram of NovaBank

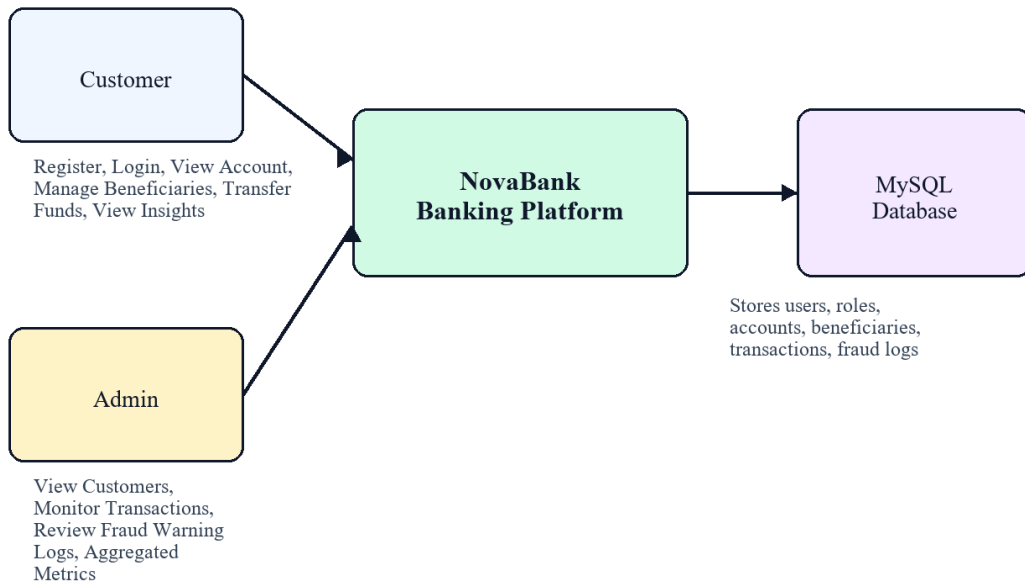


Figure 1. Context Diagram of NovaBank

Figure 2. Layered System Architecture of NovaBank

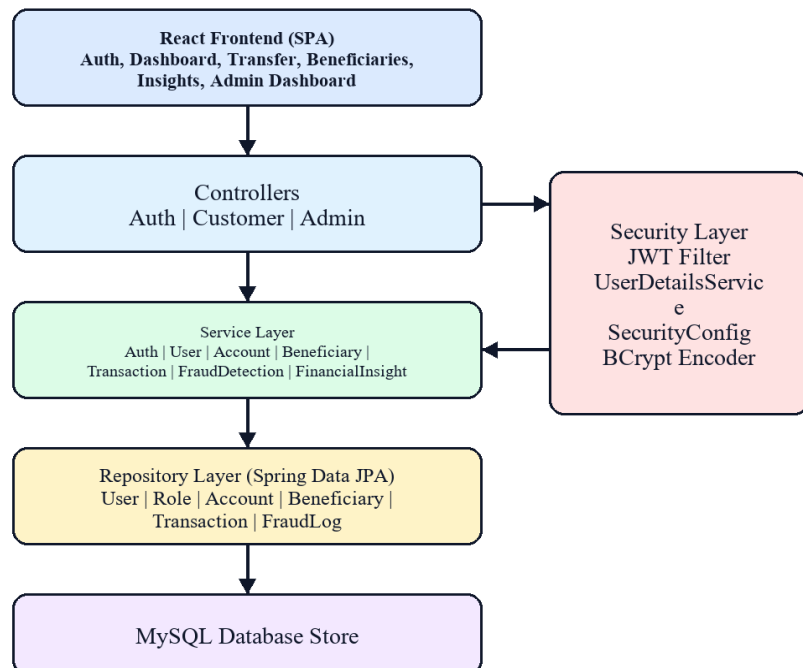


Figure 2. Layered System Architecture of NovaBank

3. Technologies Used

The technology stack is selected to mirror modern enterprise environments, focusing on framework reliability, package support, security, and developer productivity. The selected stack is summarized below:

Table 1. Technology Stack and Frameworks Used

Layer	Technologies / Tools
Frontend	React 18, Vite, Tailwind CSS, Axios, React Router v6, Chart.js
Backend	Java 17, Spring Boot 3.x, Spring Security 6.x, JWT (JWT library), Spring Data JPA, Maven
Database	MySQL 8.0, Hibernate ORM
API Testing & Documentation	Swagger UI via SpringDoc OpenAPI, Postman
Design Approach	Layered client-server MVC, Modular monolith design pattern

4. Front-end Development

The frontend application is engineered as a responsive Single Page Application (SPA). React 18 is used to build reusable component trees, while Tailwind CSS serves as the styling framework, ensuring a clean, modern user experience. Navigation and route protection are handled using React Router v6.

To ensure code reusability and maintainability, frontend components are organized into standard packages. 'AppShell' provides the main layout, side navigation, and user context headers. 'ProtectedRoute' intercepts route changes, redirecting unauthenticated traffic to the login view and checking role permissions. Reusable elements like 'DataTable', 'FormInput', 'StatCard', and 'RiskBadge' standardize UI appearance across dashboards.

The application dashboard integrates financial data visualization. The dashboard home page features account balance summaries, quick transfer utilities, recent activity tables, and visual charts showing monthly credit and debit trends. The user interface adapts dynamically to desktop, tablet, and mobile screens, providing a consistent responsive layout.

Figure 3. Use Case Diagram of NovaBank

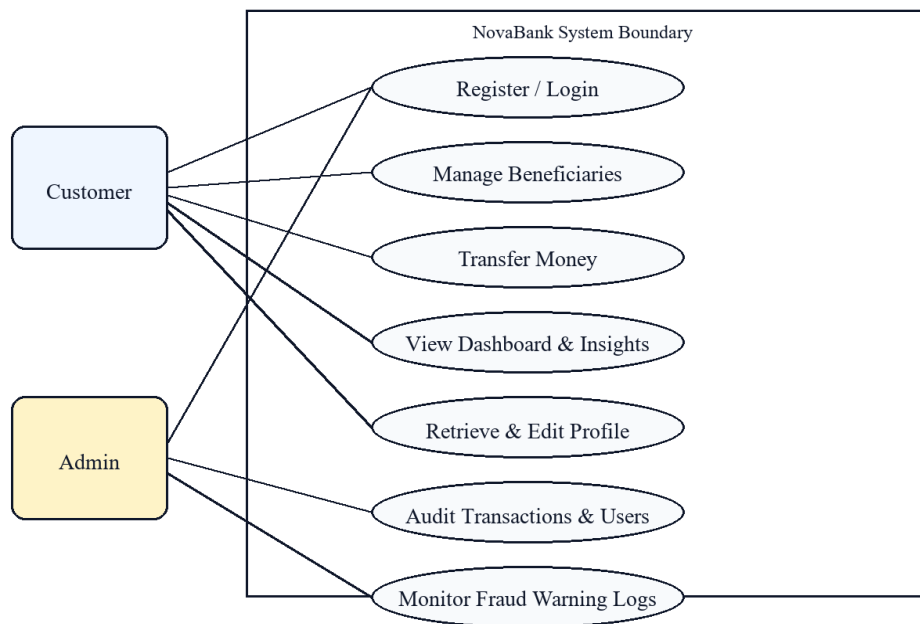


Figure 3. Use Case Diagram of NovaBank

5. Back-end Development

The backend is developed as a modular Spring Boot REST API. Spring Security 6 is integrated with stateless authentication, using custom security filters to intercept incoming requests and validate JWT tokens before controller actions are executed. Centralized exception handling intercepts errors, returning standardized JSON responses.

Business logic is encapsulated in the service layer, keeping controllers focused on payload mapping and validation. REST inputs are validated using standard Java Bean Validation annotations (e.g. `@NotBlank`, `@Email`, `@Positive`). DTO (Data Transfer Object) projections ensure database entities are never exposed raw, preventing data leaks.

The backend also implements the data provisioning lifecycle. When a new customer registers, the system hashes the password, creates a user profile, and automatically provisions a new checking account with a unique 12-digit account number and a default balance. The user details are stored securely, making the account ready for immediate transactions.

Table 2. Major Application Modules

Module	Description
Authentication Module	Handles registration, user login, credential verification, and JWT token issuance.

Module	Description
Customer Module	Manages user profiles, account summaries, transaction history, and savings metrics.
Admin Module	Aggregates platform KPIs, lists customer profiles, audits all transactions, and views fraud logs.
Transfer Module	Executes secure fund transfers, manages balances atomic updates, and records ledger entries.
Fraud Engine	Inspects transfer parameters, calculates risk scores, and writes fraud warning entries.
Security Module	Configures CORS, configures endpoint filters, decodes JWT, and hashes passwords via BCrypt.

6. Database Design

The relational database schema is designed in 3rd Normal Form (3NF) to eliminate data redundancy and ensure referential integrity. Seven core tables manage the application data: users, roles, user_roles, accounts, beneficiaries, transactions, and fraud_logs.

A key design choice is separating the 'users' and 'accounts' tables, allowing support for multiple banking products under a single user profile. Transactions are recorded in a ledger-style, double-entry format. Every account-to-account transfer creates two transaction records (a DEBIT type record for the sender and a CREDIT type record for the receiver) to guarantee complete audit trails.

MySQL 8.0 is selected as the Database Management System. This decision is justified by MySQL's reliable support for ACID transactions, robust foreign key constraint enforcement, and built-in locking capabilities (pessimistic and optimistic locks), which are essential for maintaining financial data consistency.

Table 3. Core Database Tables and Purpose

Table	Purpose
users	Stores customer/admin identity details, contact info, hashed passwords, and active state.
roles	Stores role names (e.g. ROLE_CUSTOMER, ROLE_ADMIN) for access control list mapping.
user_roles	Join table mapping users to roles (many-to-many relationship).
accounts	Stores account number, balance, currency, active flag, and links to user (one-to-one).

Table	Purpose
beneficiaries	Stores saved payee details: name, account number, bank, IFSC code, nickname, and user ID.
transactions	Stores transfer records: reference number, type, amount, post-balance, and sender/receiver accounts.
fraud_logs	Stores risk assessment details: risk score, risk level, evaluation rules violated, and transaction link.

Figure 4. Database Entity Relationship Diagram of NovaBank

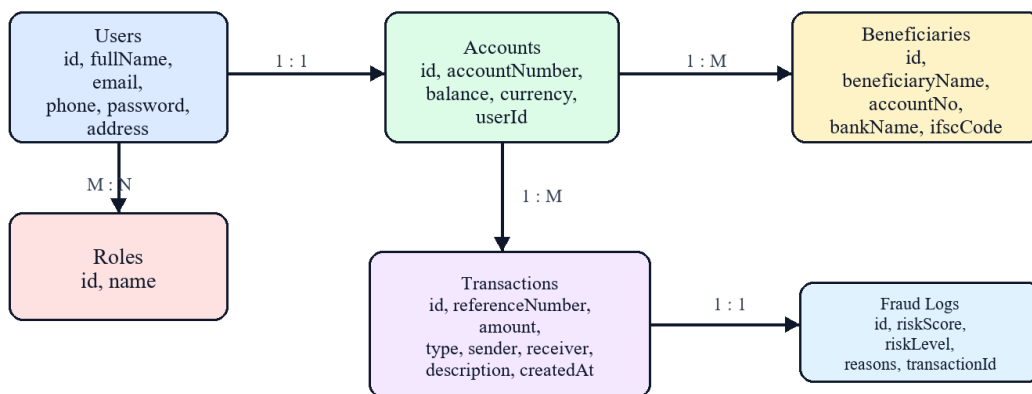


Figure 4. Database Entity Relationship Diagram of NovaBank

7. Authentication and Security

Security is the core component of NovaBank. Public endpoints (e.g., login, registration) are exposed, while all business endpoints are protected behind a Spring Security filter chain. Passwords are never stored in plain text; instead, they are hashed using the BCrypt algorithm with a cost factor of 10 during registration.

Authorization is role-based. Endpoints prefixed with '/api/admin/**' require the ROLE_ADMIN role, whereas endpoints under '/api/customer/**' are restricted to users with the ROLE_CUSTOMER role. On login, the system signs a JWT containing the username and roles. The client sends this token in the Authorization header. On the server, a custom JwtAuthenticationFilter validates the token and sets the security context.

Data protection is also enforced during API communication. DTOs screen database entities, ensuring fields like password hashes, account IDs, and system keys are excluded from responses. The system also configures strict Cross-Origin Resource Sharing (CORS) policies, allowing requests only from verified frontend origins.

8. Project Management

The capstone development followed a structured Agile plan spanning 8 weeks, with regular milestones and sprint goals to ensure complete implementation. Sprints were organized bi-weekly, starting with DB design and authentication, moving to core transactional logic, and concluding with analytics integration, frontend dashboard, and security validation.

Project risk assessment and mitigation were conducted proactively during development. Sprints were managed using Git-based branching to ensure code isolation, and Swagger UI was utilized to validate API contracts independently of frontend status. Regular integration reviews helped align frontend forms with backend data objects.

Table 4. Development Methodology Phases

Phase	Description
Requirement Analysis	Analyze digital banking workflows, finalize functional specifications, define Customer/Admin roles.
System Design	Create database schema, design API endpoint contracts, define class structure and sequence flows.
Backend Development	Implement Spring Boot API, write database repositories, integrate Spring Security, JWT, and exceptions.
Frontend Integration	Develop responsive pages in React, connect backend endpoints via Axios, secure frontend routes.
Validation & Launch	Test concurrent transaction safety, verify fraud rules, audit security constraints, document final code.

Table 5. Project Implementation Timeline

Week	Planned Activities
Weeks 1 - 2	Requirement modeling, database schema configuration, package scaffolding, and Git repository setup.
Weeks 3 - 4	Implementation of Spring Security configuration, BCrypt hashing, JWT generation, and User authentication APIs.
Week 5	Implementation of secure transfer service with database locking, fraud risk scoring, and financial insights.

Week	Planned Activities
Week 6	Frontend design, React routing setup, AuthContext integration, and Axios request interceptors creation.
Week 7	Frontend pages development (transfer, insights, admin, profile), API integration, and Swagger verification.
Week 8	End-to-end testing, validation of pessimistic locking under load, code refactoring, and final report preparation.

Table 6. Risk Assessment and Mitigation Matrix

Risk Area	Possible Impact	Mitigation Strategy
Payload Mismatch	Differences between React data models and Spring Boot DTO objects lead to API failures.	Document API contracts using OpenAPI/Swagger and auto-validate schema models during testing.
Concurrency Conflicts	Concurrent transfers to the same account cause race conditions and balance inaccuracy.	Implement pessimistic write locking (@Lock) on database select queries to serialize balance updates.
Token Hijacking	JWT tokens are intercepted, allowing unauthorized access to banking operations.	Set token expiry limits, transmit tokens exclusively via HTTPS, and validate tokens in an intercepting filter.
Scope Creep	Extended requirements like external bank integrations delay target completion.	Freeze the core feature set early, focusing on robust internal transfers and simulated external gateways.

9. Results and Evaluation

NovaBank has been successfully implemented and verified against all functional and security goals. Unit tests and manual verification using Swagger UI confirm that the REST API behaves correctly. All endpoints, including registration, profile update, transfer execution, fraud auditing, and financial analytics, return correct JSON responses and appropriate HTTP status codes.

To evaluate transaction safety under high concurrency, parallel threads were simulated to initiate transfers to and from the same account. Without database locks, a race condition occurred, leading to mismatched balances. Integrating Spring Data JPA's Pessimistic Write Lock (@Lock(LockModeType.PESSIMISTIC_WRITE)) on the account lookup query resolved this issue. This forced database queries to serialize, preventing double-spending and ensuring database consistency.

Performance evaluation demonstrates that the application is highly efficient. The stateless JWT architecture reduces server memory overhead. Database indices on indexed columns (like email, account number, and transaction references) ensure SQL query response times remain under 50ms, while React components re-render optimally using local state hooks.

Table 7. Capstone Functionality Implementation Coverage

Functionality	Status	Implementation Detail
Authentication Flow	Fully Implemented	User registration, password hashing via BCrypt, JWT generation, role guards.
Account Provisioning	Fully Implemented	New customers receive a checking account with a unique 12-digit number and balance.
Beneficiary CRUD	Fully Implemented	Customers can add, edit, view, and delete saved payee records.
Atomic Transfers	Fully Implemented	Pessimistic locking prevents concurrent balance overwrite; updates occur in one transaction.
Fraud Engine	Fully Implemented	Classifies transactions into LOW, MEDIUM, HIGH risk based on rules; writes logs.
Financial Insights	Fully Implemented	Calculates monthly credit/debit totals, savings rate, and yields personalized advice.
Admin Dashboard	Fully Implemented	Displays system-wide statistics (KPIs), views all transactions, and views fraud alerts.
Profile Management	Fully Implemented	Customers can retrieve and update address, phone, and name details via safe APIs.

10. Conclusion

The NovaBank project demonstrates the successful design and implementation of a modern, secure full-stack banking portal. The project integrates core web standards, Spring Security controls, database synchronization safeguards, and rule-based data analytics into a functional Single Page Application.

Key takeaways include: the importance of DTO design in preventing data leaks, the need for pessimistic locking to prevent race conditions during concurrent financial operations, and the value of structured Agile planning in delivering complex deliverables on time. The layered codebase design ensures the app remains maintainable and scalable.

Recommended future updates include: integrating OAuth2 for third-party authentication, adding email/SMS alerts for high-risk transactions, implementing machine-learning-based

fraud scoring trained on historic banking datasets, and deploying the application onto Docker containers on cloud environments.

ANNEXURE A. KEY IMPLEMENTED REST ENDPOINTS

The following API summaries document the endpoint contracts developed to connect the React front-end pages to the Spring Boot REST services:

Table 8. REST API Endpoint Catalog

Endpoint Pattern	Access	Detailed Purpose
POST /api/auth/register	Public	Customer registration; Provisions user record and banking account.
POST /api/auth/login	Public	Authenticates login credentials; returns token, role, and profile details.
GET /api/customer/profile	Customer/Admin	Retrieves current authenticated user's profile details.
PUT /api/customer/profile	Customer	Updates personal profile (name, phone, address) details.
GET /api/customer/account	Customer/Admin	Fetches account summaries: account number, balance, currency.
POST /api/customer/transfer	Customer	Executes atomic account-to-account money transfer under transaction locks.
GET /api/customer/transactions	Customer/Admin	Retrieves transaction history, with description/reference filtering.
GET /api/customer/insights	Customer	Fetches monthly credits, debits, savings rate, and financial tips.
GET /api/admin/dashboard	Admin	Retrieves overall metrics: total users, transactions, volume, and fraud logs count.
GET /api/admin/customers	Admin	Lists all system customers with profiles and account status, excluding passwords.
GET /api/admin/fraud-logs	Admin	Lists system-wide transaction fraud warnings for operational auditing.

ANNEXURE B. ENTITY-TO-MODULE MAPPINGS

This annexure links system entity classes with the corresponding application modules:

Table 9. Database Entity to Application Module Mapping

Entity Class	Module Category	Functional Responsibility
User + Role	Authentication & Profile	Manages user credentials, profile information, and access control mappings.
Account	Account & Balance	Tracks provisioned banking accounts, balances, currencies, and user ownership.
Beneficiary	Beneficiary CRUD	Manages saved payees that the customer can select for fund transfers.
BankTransaction	Ledger Transactions	Stores immutable records of individual debits and credits for all transfers.
FraudLog	Fraud Auditing	Records security warnings and risk scores generated by the fraud detection engine.

ANNEXURE C. SYSTEM DESIGN DIAGRAMS

The following detailed diagrams demonstrate system workflows, lifecycle execution, and server deployment boundaries:

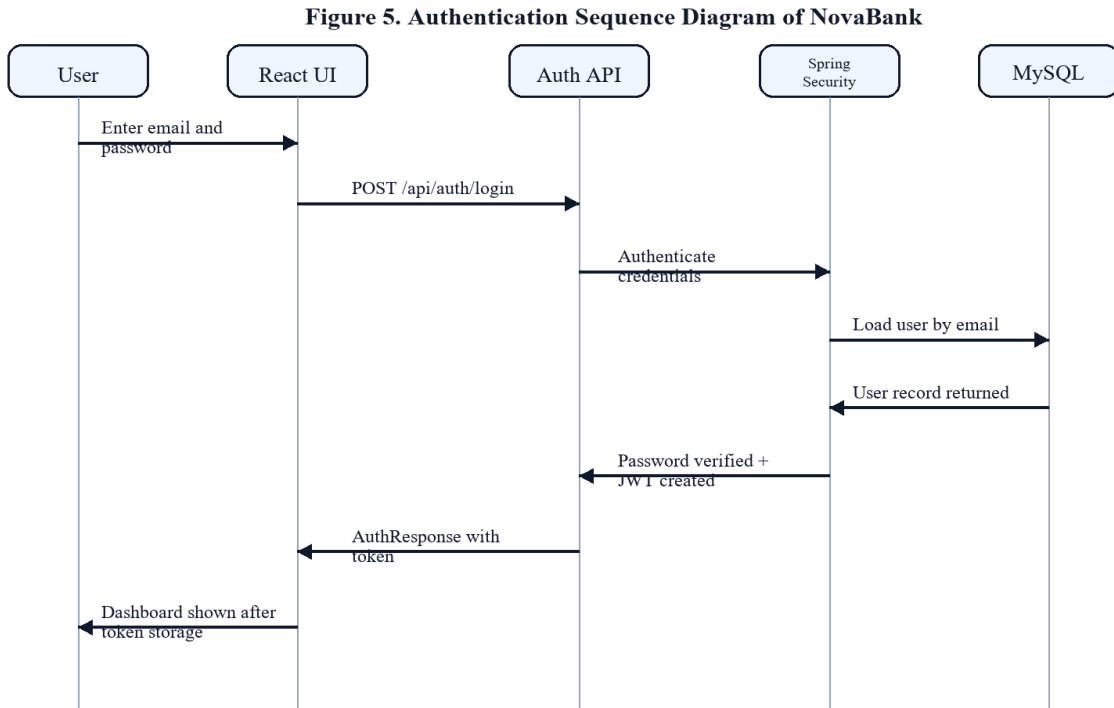


Figure 5. Authentication Sequence Diagram of NovaBank

Figure 6. Beneficiary Management Activity Diagram of NovaBank

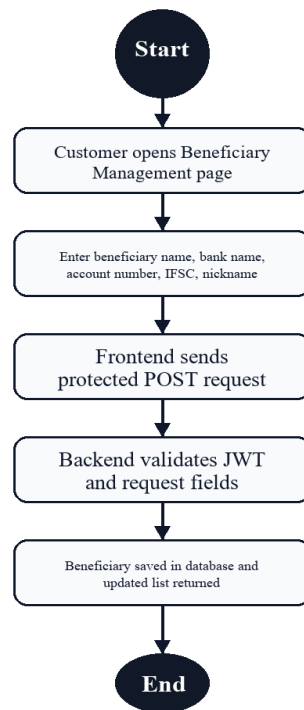


Figure 6. Beneficiary Management Activity Diagram of NovaBank

Figure 7. Deployment and Runtime View of NovaBank

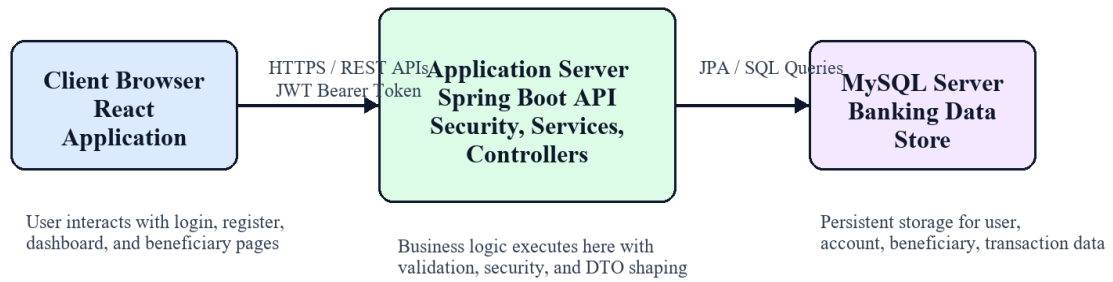


Figure 7. Deployment and Runtime View of NovaBank